



机器学习

基本概念

定义 通过对数据的优化学习，建立能够刻画数据中所蕴含语义概念或分布结构等信息的模型

没有免费午餐定理：任何一个机器学习模型如果在一些训练集以外的样本误差小（off-training set error），那么必然在另外一些训练集以外的样本上表现欠佳，任何模型在平均意义上而言其性能都是一样的，即没有放之四海而皆准的最好算法

损失函数 $Loss(f(x_i), y_i)$ ：估量预测值 $\hat{y}_i$ 和真实值 y_i 之间的差异；在训练过程中希望在训练集上累计差异（损失）最小

表 4.1 常见损失函数的定义

损失函数名称	损失函数定义
0-1 损失函数	$Loss(y_i, f(x_i)) = \begin{cases} 1, & f(x_i) \neq y_i \\ 0, & f(x_i) = y_i \end{cases}$
平方损失函数	$Loss(y_i, f(x_i)) = (y_i - f(x_i))^2$
绝对损失函数	$Loss(y_i, f(x_i)) = y_i - f(x_i) $
对数损失函数 / 对数似然函数	$Loss(y_i, P(y_i x_i)) = -\log P(y_i x_i)$

经验风险 R_{emp} ：在训练集上所产生的损失

- 经验风险越小说明模型对训练集数据拟合程度越好
- $\frac{1}{n} \sum_{i=1}^n Loss(y_i, f(x_i))$

期望风险 R ：在所有数据上产生的损失

- 记该任务中所有数据的联合分布为 $P(x, y)$
- $\int_{x \times y} Loss(y, f(x)) P(x, y) dx dy$

机器学习的目标：经验风险最小化

- $R \leq R_{emp} + err$
- err 取值与机器学习模型的复杂程度和训练集样本数目有关
- 过拟合：使用同一批训练数据反复训练，模型会变得复杂，此时 R_{emp} 降低，但 err 变大，导致 R 变大

泛化能力：模型在训练集上所取得性能与在测试集上所取得性能保持一致的能力

经验风险	期望风险	模型泛化能力
经验风险小 (训练集上表现好)	期望风险小 (所有数据上表现好)	泛化能力强
经验风险小 (训练集上表现好)	期望风险大 (所有数据上表现不好)	过学习 (模型过于复杂)
经验风险大 (训练集上表现不好)	期望风险大 (所有数据上表现不好)	欠学习
经验风险大 (训练集上表现不好)	期望风险小 (所有数据上表现好)	“神仙算法”或“黄粱美梦”

正则化：防止过拟合，降低模型复杂度

- 修改经验风险为 $\frac{1}{n} \sum_{i=1}^n Loss(y_i, f(x_i)) + \lambda J(f)$
- $J(f)$ ：正则化因子或惩罚项因子
- λ ：调整惩罚强度的系数

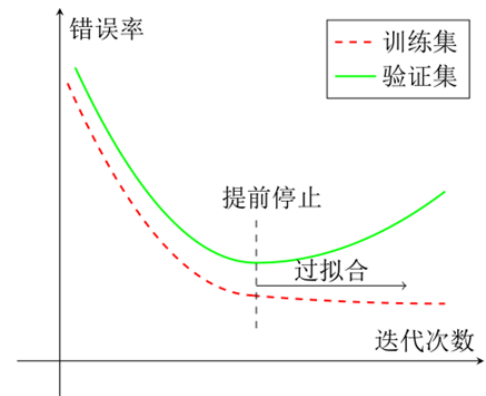
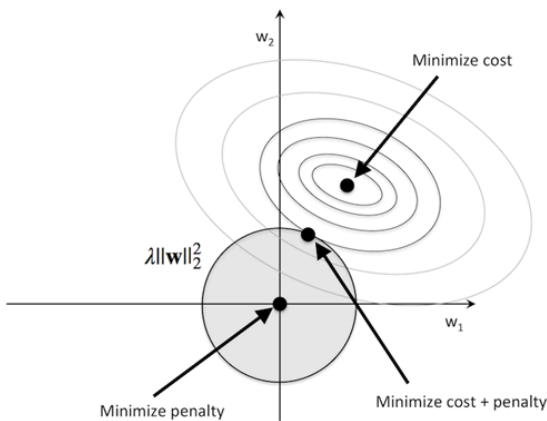
所有损害优化的方法都是正则化

增加优化约束

L1/L2约束、数据增强

干扰优化过程

权重衰减、随机梯度下降、提前停止



模型度量方法

以二分类为例，设 n 为训练样例的总数，正例总数和负例总数分别是 P (positive)和 N (negative)

		预测情况	
		正例 (Positive)	反例 (Negative)
真实情况	正例 (Positive)	TP (True Positive)	FN (False Negative)
	反例 (Negative)	FP (False Positive)	TN (True Negative)

- 准确率: $ACC = \frac{TP+TN}{P+N}$
- 错误率: $errorRate = \frac{FP+FN}{P+N}$
- 查准率/精确率: $precision = \frac{TP}{TP+FP}$, 被预测为正例中实际也为正例的比例
- 查全率/召回率: $recall = \frac{TP}{TP+FN}$, 实际为正例中被预测为正例的比例
- 实际应用中, 精确率和召回率之间是相互矛盾的

- 综合二者，有 $F1 - score = 2 \times \frac{precision \times recall}{precision + recall}$ ，为二者的调和平均数

参数优化

- 频率学派：
 - 对模型参数优化学习的结果就是得到使观测数据发生概率最大的模型参数，又称为最大似然估计
 - 通过调整模型参数使得模型能够最大化样本情况出现的概率
- 贝叶斯学派：
 - 对模型参数优化学习的结果就是似然概率（模型参数产生数据的概率）与先验概率（没有任何实验数据时对模型参数的经验判断）乘积最大，又称为最大后验估计
 - 最大化在给定数据样本的情况下模型参数的后验概率

有监督学习

基本概念

目标：给定带有标签信息数据的训练集 $D = \{(x_i, y_i)\}_{i=1}^n$ ，学习一个从输入 x_i 到输出 y_i 的映射
映射函数 f ：将输入数据映射到语义标注空间，实现数据的分类和识别

训练集：用于模型训练

验证集：用于评估模型以调整参数

测试集：用于得到模型的优劣水平

回归分析

定义

分析不同变量之间存在关系的研究叫作回归分析，刻画不同变量之间关系的模型称为回归模型

分类

- 一元/多元
- 线性/非线性

线性回归作用

- 预测：根据输入变量预测输出变量的值

- 解释关系：通过模型参数，解释输入变量与输出变量之间的关系
- 数据建模：将实际数据拟合到线性模型中，以便更好地理解和分析数据

一元线性回归模型

目标：寻找一条用 $y = ax + b$ 表达的直线，使得这条直线尽可能靠近或穿过给定数据，即能够以最小误差来拟合给定数据

求解过程

1. 模型与目标函数

- **模型形式：** $\hat{y}_i = ax_i + b$
- **残差平方和（损失函数）：**

$$L(a, b) = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \sum_{i=1}^n (y_i - ax_i - b)^2$$

- **目标：**最小化 $L(a, b)$

2. 求解截距 b

- **对 b 求偏导并令导数为零：**

$$\frac{\partial L}{\partial b} = -2 \sum_{i=1}^n (y_i - ax_i - b) = 0$$

- **化简：**

$$\sum_{i=1}^n y_i - a \sum_{i=1}^n x_i - nb = 0$$

- **解得 b ：**

$$b = \bar{y} - a\bar{x}$$

- 其中, $\bar{x}$ 和 $\bar{y}$ 分别是 x 和 y 的样本均值

3. 求解斜率 a

- **对 a 求偏导并代入 b ：**

$$\frac{\partial L}{\partial a} = -2 \sum_{i=1}^n (y_i - ax_i - b)x_i = 0$$

- 代入 $b = \bar{y} - a\bar{x}$, 化简后得到:

$$a = \frac{\sum_{i=1}^n (x_i y_i) - n\bar{x}\bar{y}}{\sum_{i=1}^n x_i^2 - n\bar{x}^2}$$

- 用协方差和方差表示:

$$a = \frac{\text{Cov}(x, y)}{\text{Var}(x)} = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}$$

4. 关键结论

- $a = \frac{\text{Cov}(x, y)}{\text{Var}(x)} = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}$
- $b = \bar{y} - a\bar{x}$

多元线性回归模型

1. 问题定义

给定 m 个训练样本 $\{(\mathbf{x}_i, y_i)\}_{i=1}^m$, 其中:

- 每个样本 $\mathbf{x}_i = [x_{i,1}, x_{i,2}, \dots, x_{i,D}] \in \mathbb{R}^D$ 是一个 D 维特征向量
- 目标是找到一组参数 $\mathbf{a} = [a_0, a_1, \dots, a_D]$, 使得线性模型:

$$f(\mathbf{x}_i) = a_0 + \sum_{j=1}^D a_j x_{i,j} = a_0 + \mathbf{a}^T \mathbf{x}_i$$

能够最小化**均方误差 (MSE)** :

$$J_m(\mathbf{a}) = \frac{1}{m} \sum_{i=1}^m (y_i - f(\mathbf{x}_i))^2$$

2. 矩阵表示

为了简化计算, 引入**增广矩阵**:

- 将每个样本 $\mathbf{x}_i$ 扩展一个维度, 使其变为 $[1, x_{i,1}, \dots, x_{i,D}]$ (对应截距 a_0)
- 数据矩阵 X ($(D+1) \times m$ 维) :

$$X = \begin{bmatrix} 1 & 1 & \cdots & 1 \\ x_{1,1} & x_{2,1} & \cdots & x_{m,1} \\ \vdots & \vdots & \ddots & \vdots \\ x_{1,D} & x_{2,D} & \cdots & x_{m,D} \end{bmatrix}$$

- 标签向量 $\mathbf{y}$ ($m \times 1$ 维) :

$$\mathbf{y} = [y_1, y_2, \dots, y_m]^T$$

- 均方误差的矩阵形式:

$$J_m(\mathbf{a}) = \frac{1}{m}(\mathbf{y} - X^T \mathbf{a})^T (\mathbf{y} - X^T \mathbf{a})$$

3. 求解最优参数 $\mathbf{a}$

目标是找到 $\mathbf{a}$ 使得 $J_m(\mathbf{a})$ 最小化:

1. 计算梯度:

$$\nabla J(\mathbf{a}) = -2X(\mathbf{y} - X^T \mathbf{a})$$

2. 令梯度为零 (因为 $J_m(\mathbf{a})$ 是凸函数, 唯一极小值点即最小值点) :

$$-2X(\mathbf{y} - X^T \mathbf{a}) = 0$$

3. 整理得到正规方程 (Normal Equation) :

$$XX^T \mathbf{a} = X\mathbf{y}$$

4. 闭式解 (解析解) :

$$\mathbf{a} = (XX^T)^{-1}X\mathbf{y}$$

(前提: XX^T 可逆)

4. 关键结论

- XX^T 可逆时, $\mathbf{a} = (XX^T)^{-1}X\mathbf{y}$

决策树

基本概念

信息熵

- 假设有 K 种信息，其组成了集合样本 D ，记第 k 种信息发生的概率为 $p_k (1 \leq k \leq K)$
- $Ent(D) = -\sum_{k=1}^K p_k \log_2 p_k$
- $Ent(D)$ 越小， D 包含的信息越确定

信息增益

- 选择属性划分样本集前后信息熵的减少量
- 描述了样本集复杂度（不确定性）所减少的程度
- $Gain(D, A) = Ent(D) - \sum_{i=1}^n \frac{|D_i|}{|D|} Ent(D_i)$

决策过程

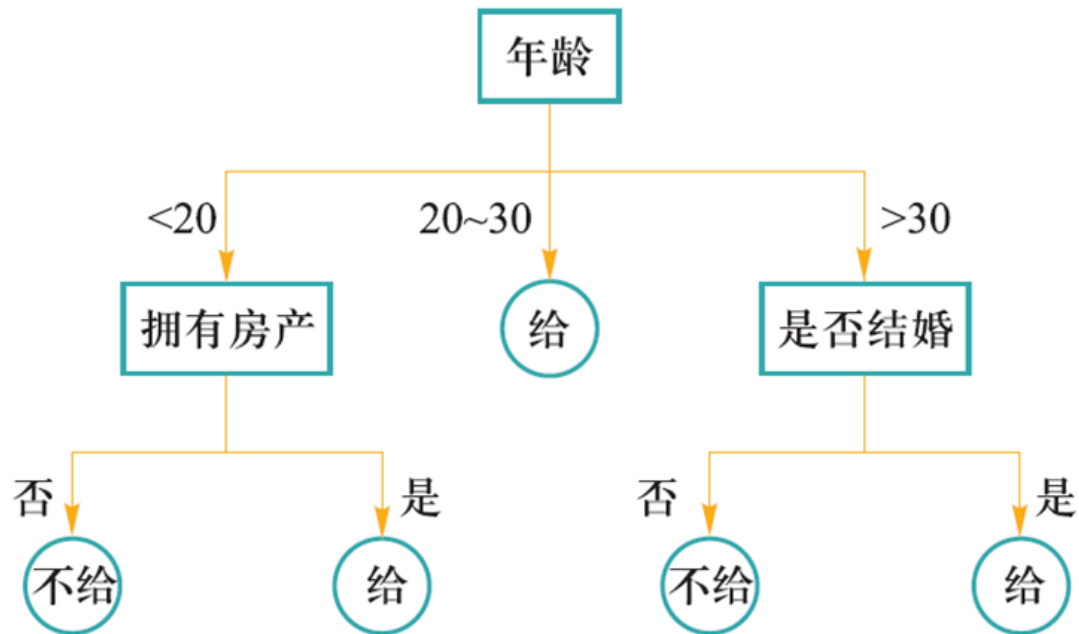
1. 分别计算按每种属性划分后带来的信息增益
2. 选择信息增益最大的属性对原样本集进行划分
3. 如果划分后每个子集都只存在同类样本，则停止划分；否则继续对子集重复上述操作

来练练手吧

表 4.5 是否给予贷款与申请人自身状况的关系

序号	年龄 / 岁	银行流水	是否结婚	是否拥有房产	是否给予贷款
1	> 30	高	否	是	否
2	> 30	高	否	否	否
3	20~30	高	否	是	是
4	< 20	中	否	是	是
5	< 20	低	否	是	是
6	< 20	低	是	否	否
7	20~30	低	是	否	是
8	> 30	中	否	是	否
9	> 30	低	是	是	是
10	< 20	中	否	是	是
11	> 30	中	是	否	是
12	20~30	中	否	否	是
13	20~30	高	是	是	是
14	< 20	中	否	否	否

请根据以上数据，手搓决策树



无监督学习

基本概念

从没有“输入-输出”关系对的数据中学习模式/特征

- 数据仅有输入，没有输出
- 没有标签/没有目标

聚类算法：将数据集中的对象划分为若干个类别或簇，使得同一类别内的对象相似度较高，不同类别之间的相似度较低

聚类算法

目标：将 n 个 d 维数据 $x_i, i = 1..n$ 划分为 K 个聚簇，最小化类内距离（，最大化类间距离

k-means算法

通过最小化聚簇内的数据方差来实现最大化类内相似度

算法步骤

1. 初始化：随机选择 K 个数据点作为初始簇中心
2. 分配：将每个数据点分配给最近的簇中心
3. 更新：重新计算每个簇的中心（即簇内所有点的均值）
4. 迭代：重复步骤 2 和 3 直到簇中心不再发生变化或达到预设的迭代次数

缺点：

- 无法保证全局最优
- 容易受初始值影响
- 可以用不同初始值重复，以达到更好的“局部”最优

```

def kmeans(data, k, max_iterations=100):
    # 步骤1: 初始化 - 随机选择k个数据点作为初始簇中心
    centers = initialize_centers(data, k)

    for iteration in range(max_iterations):
        # 步骤2: 分配 - 将每个数据点分配给最近的簇中心
        labels = assign_clusters(data, centers)

        # 步骤3: 更新 - 重新计算每个簇的中心
        new_centers = update_centers(data, labels, k)

        # 检查收敛条件: 如果簇中心不再变化, 则停止迭代
        if convergence_check(centers, new_centers):
            break

        # 更新簇中心
        centers = new_centers

    return labels, centers

def initialize_centers(data, k): //随机选择k个数据点作为初始簇中心
    indices = random.sample(range(len(data)), k)
    centers = [data[i] for i in indices]
    return centers

def assign_clusters(data, centers): //将每个数据点分配给最近的簇中心
    labels = []
    for point in data:
        # 计算点到每个簇中心的距离
        distances = [distance(point, center) for center in centers]
        # 找到最近的簇中心
        closest_center_index = np.argmin(distances)
        labels.append(closest_center_index)
    return labels

def update_centers(data, labels, k): //重新计算每个簇的中心 (簇内所有点的均值)
    new_centers = []
    for i in range(k):
        # 获取属于当前簇的所有点

```

```
cluster_points = [point for point, label in zip(data, labels) if label == i]
# 计算簇内点的均值作为新的簇中心
new_center = np.mean(cluster_points, axis=0)
new_centers.append(new_center)
return new_centers
```

特征降维

线性判别分析

对于一组具有标签信息的高维数据样本，LDA利用其类别信息，将其线性投影到一个低维空间上，在低维空间中同一类别样本尽可能靠近，不同类别样本尽可能彼此远离

数学定义

- **数据集**: $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$, 其中 $\mathbf{x}_i \in \mathbb{R}^d$, 标签 $y_i \in \{\mathcal{C}_1, \mathcal{C}_2, \dots, \mathcal{C}_K\}$ (共 K 类)
- **关键统计量**:
 - $\mathbf{m}$: 全体样本均值向量
 - $\mathbf{m}_i$: 第 i 类样本的均值向量
 - Σ_i : 第 i 类样本的协方差矩阵:

$$\Sigma_i = \sum_{\mathbf{x} \in \mathcal{C}_i} (\mathbf{x} - \mathbf{m}_i)(\mathbf{x} - \mathbf{m}_i)^T$$

二分类问题 ($K = 2$)

- **投影函数**: $y(\mathbf{x}) = \mathbf{w}^T \mathbf{x}$ (投影到一维空间)
- **优化目标**: 最大化类间散度与类内散度的比值 (Fisher 准则):

$$J(\mathbf{w}) = \frac{\text{类间散度}}{\text{类内散度}} = \frac{\mathbf{w}^T \mathbf{S}_b \mathbf{w}}{\mathbf{w}^T \mathbf{S}_w \mathbf{w}}$$

- **类间散度矩阵** (Between-class scatter):

$$\mathbf{S}_b = (\mathbf{m}_2 - \mathbf{m}_1)(\mathbf{m}_2 - \mathbf{m}_1)^T$$

- **类内散度矩阵** (Within-class scatter):

$$\mathbf{S}_w = \Sigma_1 + \Sigma_2$$

- **求解最优投影方向：**

- 通过拉格朗日乘子法，转化为广义特征值问题：

$$\mathbf{S}_w^{-1} \mathbf{S}_b \mathbf{w} = \lambda \mathbf{w}$$

- 最优解为：

$$\mathbf{w} = \mathbf{S}_w^{-1} (\mathbf{m}_2 - \mathbf{m}_1)$$

(忽略缩放因子，仅保留方向)

多分类问题 ($K \geq 2$)

- **扩展目标：**投影到 $K - 1$ 维空间 (因秩限制)
- **全局类内散度矩阵：**

$$\mathbf{S}_w = \sum_{i=1}^K \boldsymbol{\Sigma}_i$$

- **全局类间散度矩阵：**

$$\mathbf{S}_b = \sum_{i=1}^K N_i (\mathbf{m}_i - \mathbf{m})(\mathbf{m}_i - \mathbf{m})^T$$

(N_i 为第 i 类样本数)

- **优化目标：**

$$\max_{\mathbf{W}} \frac{\text{tr}(\mathbf{W}^T \mathbf{S}_b \mathbf{W})}{\text{tr}(\mathbf{W}^T \mathbf{S}_w \mathbf{W})}$$

- 解为 $\mathbf{S}_w^{-1} \mathbf{S}_b$ 的前 $K - 1$ 个最大特征值对应的特征向量组成的投影矩阵 $\mathbf{W}$

算法流程

1. **输入：**数据矩阵 $\mathbf{X}$ 和标签向量 $\mathbf{y}$
2. **计算统计量：**
 - 各类均值 $\mathbf{m}_i$ 和全局均值 $\mathbf{m}$
 - 类内散度 $\mathbf{S}_w$ 和类间散度 $\mathbf{S}_b$

3. **求解投影矩阵:**

- 计算 $S_w^{-1} S_b$ 的特征向量
- 取前 $K - 1$ 个特征向量构成 W

4. **降维:** $X_{\text{LDA}} = XW$

```

def linear_discriminant_analysis(X, y, n_components):

# 步骤1: 计算类内散度矩阵 (Sw) 和类间散度矩阵 (Sb)
    Sw = compute_within_class_scatter(X, y)
    Sb = compute_between_class_scatter(X, y)

# 步骤2: 计算  $Sw^{-1} * Sb$  的特征值和特征向量
    eigenvalues, eigenvectors = np.linalg.eig(np.linalg.inv(Sw) @ Sb)

# 步骤3: 按特征值从大到小排序, 选择前 n_components 个特征向量
    sorted_indices = np.argsort(eigenvalues)[::-1]
    W = eigenvectors[:, sorted_indices[:n_components]]

# 步骤4: 将数据投影到低维空间
    X_lda = X @ W

    return W, X_lda

def compute_within_class_scatter(X, y):
    """
    计算类内散度矩阵 Sw
     $Sw = \sum (\sum (x - \mu_i)(x - \mu_i)^T)$  for each class i
    """
    n_features = X.shape[1]
    Sw = np.zeros((n_features, n_features))
    classes = np.unique(y)

    for c in classes:
        X_c = X[y == c]
        mean_c = np.mean(X_c, axis=0)
        Sw += (X_c - mean_c).T @ (X_c - mean_c)

    return Sw

def compute_between_class_scatter(X, y):
    """
    计算类间散度矩阵 Sb
     $Sb = \sum n_i * (\mu_i - \mu)(\mu_i - \mu)^T$  for each class i
    """

```



```

n_features = X.shape[1]
Sb = np.zeros((n_features, n_features))
mean_total = np.mean(X, axis=0)
classes = np.unique(y)

for c in classes:
    X_c = X[y == c]
    n_c = X_c.shape[0]
    mean_c = np.mean(X_c, axis=0)
    Sb += n_c * (mean_c - mean_total).reshape(-1, 1) @ (mean_c - mean_total).reshape(1, -1)

return Sb

```

主成分分析

主要思想

- 通过分析找到数据特征的主要成分，使用这些主要成分来代替原始数据
- 将 d 维特征数据映射到 l 维空间（一般 $d \gg l$ ），去除原始数据之间的冗余性
- 将原始数据向方差最大的方向进行投影

算法 4.2 主成分分析

输入： n 个 d 维样本数据所构成的矩阵 X ，降维后的维数 l

输出： 映射矩阵 $W = \{w_1, w_2, \dots, w_l\}$

算法步骤：

(1) 对于每个样本数据 x_i 进行中心化处理: $x_i = x_i - \mu$, $\mu = \frac{1}{n} \sum_{i=1}^n x_j$ 。

(2) 计算原始样本数据的协方差矩阵: $\Sigma = \frac{1}{n-1} X^T X$

(3) 对协方差矩阵 Σ 进行特征值分解，对所得特征根进行排序: $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_l$ 。

(4) 取前 l 个最大特征根所对应的特征向量 $w_1, w_2, \dots, w_l$ 组成映射矩阵。

```
def pca(data, n_components):
    # 步骤1: 数据标准化
    # 计算每列的均值
    mean = np.mean(data, axis=0)
    standardized_data = data - mean

    # 步骤2: 计算协方差矩阵
    covariance_matrix = np.cov(standardized_data, rowvar=False)

    # 步骤3: 计算协方差矩阵的特征值和特征向量
    eigenvalues, eigenvectors = np.linalg.eig(covariance_matrix)
# 步骤4: 对特征值进行排序, 并选择前n_components个特征向量
    # 获取特征值的索引并按降序排序
    sorted_indices = np.argsort(eigenvalues)[::-1]
    top_eigenvectors = eigenvectors[:, sorted_indices[:n_components]]

    # 步骤5: 数据投影
    # 将标准化后的数据投影到选定的特征向量上
    reduced_data = np.dot(standardized_data, top_eigenvectors)
    return reduced_data
```

演化算法

主要思想

- 对于一个待求解的最优化问题, 先给定一定数量候选解 (称为个体或染色体), 然后让这些染色体所构成种群向最优解进化
- 在每一次进化过程中, 计算每个染色体 (即候选解) 的适应度 (fitness), 根据适应度选择合适染色体, 随后施以选择和变异来产生新的生命种群, 该种群在算法下一次迭代中被作为当前种群

算法步骤

1. 初始化具有若干规模数目的群体。当前进化代数Generation=0
2. 采用评估函数计算群体中每个染色体的适应值, 保存适应值最大的染色体Best
3. 采用轮盘赌选择算法对群体中染色体进行选择, 产生同样规模的种群
4. 按照概率从种群中选择两对染色体进行交叉操作, 即对所选中两对染色体中相应基因片段信

息进行交换。交叉所得染色体进入新种群，未交叉操作染色体被复制进入新种群

5. 按照概率对新种群中被选中染色体的若干基因片段进行变异操作。变异后染色体取代原有染色体进入新种群，未发生变异染色体直接进入新群体
6. 计算种群中各个染色体的适应值。倘若种群中染色体最大适应值大于已知Best适应值，则取代原有Best
7. 当前进化代数Generation加1。如果Generation超过规定的最大进化代数或Best达到规定的误差要求，算法结束；否则返回3

```

Generation = 0
# 步骤1: 初始化具有若干规模数目的群体。当前进化代数Generation=0
population = initialize_population(population_size, chromosome_length)
Best = None # 保存适应值最大的染色体

# 步骤2: 采用评估函数计算群体中每个染色体的适应值, 保存适应值最大的染色体Best。
fitness_scores = evaluate_population(population)
Best = population[np.argmax(fitness_scores)] # 保存适应值最大的染色体

while Generation < max_generations and not convergence_criteria(Best):

    # 步骤3: 采用轮盘赌选择算法对群体中染色体进行选择, 产生同样规模的种群。
    def roulette_wheel_selection(population, fitness_scores):
        total_fitness = sum(fitness_scores)
        probabilities = [fitness / total_fitness for fitness in fitness_scores]
        selected_indices = np.random.choice(len(population), size=len(population), p=probabilities)
        selected_population = [population[i] for i in selected_indices]
        return selected_population

    selected_population = roulette_wheel_selection(population, fitness_scores)

    # 步骤4: 按照概率从种群中选择两对染色体进行交叉操作, 定义交叉操作函数
    def crossover(parent1, parent2, crossover_rate):
        if random.random() < crossover_rate:
            crossover_point = random.randint(1, len(parent1) - 1)
            child1 = parent1[:crossover_point] + parent2[crossover_point:]
            child2 = parent2[:crossover_point] + parent1[crossover_point:]
            return child1, child2
        else:
            return parent1, parent2

    new_population = []
    for i in range(0, len(selected_population), 2):
        parent1 = selected_population[i]
        parent2 = selected_population[i+1]
        child1, child2 = crossover(parent1, parent2, crossover_rate)
        new_population.extend([child1, child2])

    # 步骤5: 按照概率对新种群中被选中染色体的若干基因片段进行变异操作

```

```

def mutate(chromosome, mutation_rate):
    mutated_chromosome = []
    for gene in chromosome:
        if random.random() < mutation_rate:
            mutated_chromosome.append(flip_gene(gene)) # 基因变异
        else:
            mutated_chromosome.append(gene)
    return mutated_chromosome

new_population = [mutate(chromosome, mutation_rate) for chromosome in new_population]

# 步骤6: 计算种群中各个染色体的适应值。倘若种群中染色体最大适应值大于已知Best适应值, 则取代原有Best。
fitness_scores = evaluate_population(new_population)
current_best_index = np.argmax(fitness_scores)
if fitness_scores[current_best_index] > evaluate_fitness(Best):
    Best = new_population[current_best_index]

# 步骤7: 当前进化代数Generation加1。如果Generation超过规定的最大进化代数或Best达到规定的误差要求, 算法结束。
Generation += 1

```